

WEB4 PHP - Game of Thrones Quotes

Nathalie Sonnleitner

Tim Peko

Inhaltsverzeichnis

1.	Einleitung	3
2.	Projektmitglieder	3
3.	Bezug zur Projektangabe	3
4.	Anwendungsüberblick	4
4.1.	Nutzerfluss nach Rolle	4
4.2.	Rollen und Berechtigungen	5
5.	Datenmodell	6
5.1.	ER-Diagramm	6
5.2.	Normalisierung und Constraints	6
5.2.1.	users	7
5.2.2.	quotes	7
5.2.3.	comments	7
5.2.4.	quote_likes	7
5.2.5.	comment_votes	8
5.3.	Testdaten	8
6.	Architektur	8
6.1.	Schichtenmodell (MVC)	8
6.2.	Komponentenübersicht	9
6.3.	Verzeichnisstruktur	10
6.4.	Ablauf: Login und Session	10
6.5.	Ablauf: Kommentar anlegen (CRUD Create)	11
6.6.	Ablauf: Kommentar löschen per REST	12
6.7.	REST-Routing	12
7.	Backend und Sicherheit	13
7.1.	PDO und Prepared Statements	13
7.2.	Validierung	13
7.3.	Schutzmaßnahmen im Überblick	13
8.	Frontend und Benutzeroberfläche	14
8.1.	HTML, CSS und JavaScript	14
8.2.	Foren- und Thread-Darstellung	14
9.	Testfälle	14
9.1.	Automatisierter REST-Testlauf (httpYac)	14
9.1.1.	Übersicht REST-Tests	15
9.1.2.	Erwartete und beobachtete Werte	15
9.2.	Manuelle UI-Tests	16
9.2.1.	Übersicht manuelle Tests	17
9.2.2.	Detaillergebnisse manuelle Tests	17
10.	Installation und Abgabe	18
10.1.	Voraussetzungen	18

10.2. Installationsschritte	18
10.3. Datenbankzugang	18
10.4. Abgabeformat	18

1. Einleitung

Diese Dokumentation beschreibt die Webanwendung **GoT Quotes**, die im Rahmen der Übung WEB4 im Sommersemester 2026 entwickelt wurde. Die Anwendung präsentiert berühmte Game-of-Thrones-Zitate als Forenbeiträge. Besucher können Zitate lesen und sortieren, eingeloggte Nutzer diskutieren in threadartigen Kommentarbäumen, und Administratoren verwalten Benutzer sowie Zitate inklusive optionaler Bild-Uploads.

Die Umsetzung folgt dem MVC-Pattern mit PHP 8.x und PDO. Für mindestens eine Ressource sind alle CRUD-Operationen implementiert: Kommentare bilden die zentrale Ressource für normale Nutzer, Zitate werden vollständig im Admin-Bereich gepflegt. Sämtliche Benutzereingaben werden serverseitig validiert, SQL-Injection und XSS werden durch Prepared Statements und konsequentes Escaping abgewehrt.

Start-URL (Standard-XAMPP): `http://localhost/`

Damit die Anwendung erreichbar ist, muss der Apache-Document-Root auf das Verzeichnis `public/` zeigen. Unter unserer Entwicklungsumgebung mit `mise` ist die Anwendung alternativ unter `http://127.0.0.1:80/` erreichbar. Der Einstiegspunkt ist stets `public/index.php`.

2. Projektmitglieder

Gemäß Projektangabe werden alle beteiligten Personen aufgeführt:

Name	Matrikelnummer
Nathalie Sonnleitner	-
Tim Peko	s2420458029

3. Bezug zur Projektangabe

Die folgende Übersicht ordnet die Pflichtanforderungen aus der Projektangabe der konkreten Umsetzung zu. Jeder Punkt ist im weiteren Verlauf dieser Dokumentation ausführlicher beschrieben.

Anforderung (Projektangabe)	Umsetzung in GoT Quotes
MVC-Pattern und CRUD für mindestens eine Ressource	Schichten Controller, Model, View; vollständiges CRUD für Kommentare und Admin-CRUD für Zitate
Rollen: Gast, eingeloggter User, Admin	Session mit <code>is_admin</code> ; unterschiedliche Navigation und Berechtigungen pro Rolle
Registrierung mit eigener Seite	<code>GET/POST /register</code> ; optionaler Avatar-Upload; Redirect zum Login ohne Auto-Login
Admin-Benutzerverwaltung	<code>/admin/users</code> : Benutzer löschen, Admin-Rolle per PATCH toggeln
PHP ≥ 8.x, PDO, saubere Schichtentrennung	Strict Types, eigener Router, Models mit ausschließlich Prepared Statements
Schutz vor SQL-Injection und XSS	PDO-Parameterbindung; <code>htmlspecialchars()</code> bei jeder HTML-Ausgabe usergenerierter Inhalte

Anforderung (Projektangabe)	Umsetzung in GoT Quotes
Serverseitige Validierung aller Eingaben	<code>ValidationService</code> prüft Formulare und Uploads; Fehler direkt am Formular oder per Flash
Self-contained ohne Composer/npm	ZIP auf XAMPP kopierbar; Tailwind CSS 4 und httpYac nur optional für Entwicklung
MySQL/MariaDB mit sinnvollem Schema	Normalisierte Tabellen, FK-Constraints, ON DELETE CASCADE bzw. SET NULL
Valides HTML5, Formatierung per CSS	Semantisches HTML5; Tailwind CSS 4 per CDN; kein Inline-Styling als Hauptmechanismus
JavaScript nicht für Validierung	<code>app.js</code> nur für Fetch (DELETE/PATCH) und UI-Helfer; Validierung ausschließlich serverseitig
ER-/UML-Diagramm, Architekturbeschreibung, Start-URL, Team	Dieses PDF-Dokument in <code>doc/</code>
Ausführliche Tests inkl. fehlerhafter Eingaben	Automatisierte httpYac-Tests plus dokumentierte manuelle UI-Tests
SQL-Dump mit CREATE DATABASE und Testdaten	<code>WEB4_PHP_TEAM7.sql</code> ; Datenbank <code>team_7</code> , User <code>fh_webphp</code>

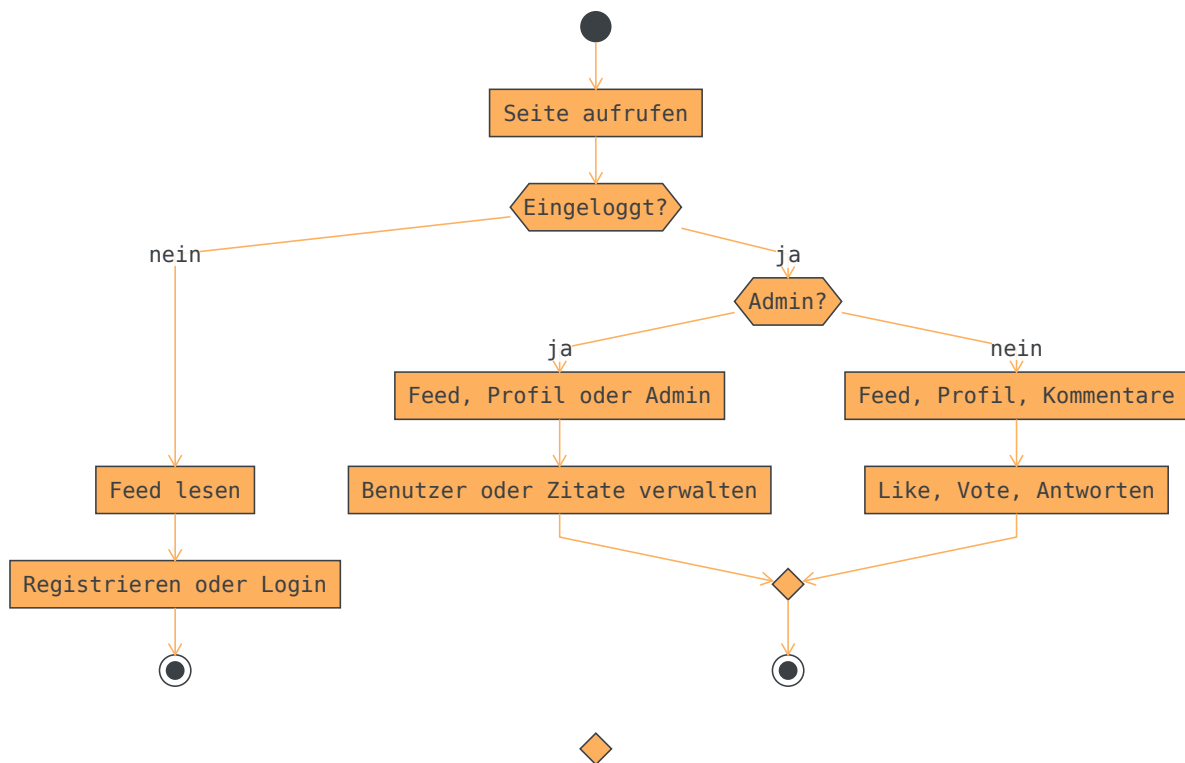
4. Anwendungsüberblick

GoT Quotes verbindet ein Zitate-Archiv mit Forenfunktionen. Jeder Datensatz in der Tabelle `quotes` entspricht einem Forenpost mit Zitattext, Sprecher, optional Staffel und Episode sowie optionalem Beitragsbild. Unter jedem Zitat entsteht eine Diskussion: Top-Level-Kommentare hängen direkt am Zitat, Antworten referenzieren über `parent_id` den jeweiligen Elternkommentar. Dadurch entsteht ein Baum beliebiger Tiefe, der in der Oberfläche eingerückt dargestellt wird.

Eingeloggte Nutzer können Zitate liken, Kommentare bewerten und ihr Profilbild pflegen. Administratoren erhalten zusätzlich Zugriff auf die Benutzerverwaltung und die Zitat-Pflege. Gäste sehen Feed, Detailseiten und öffentliche Profile, dürfen sich registrieren und einloggen, können aber weder kommentieren noch liken.

4.1. Nutzerfluss nach Rolle

Das folgende Aktivitätsdiagramm zeigt typische Pfade durch die Anwendung. Die Navigation passt sich der Session an: Gäste sehen Links zu Login und Registrierung, eingeloggte Nutzer zu Profil und Logout, Admins zusätzlich Einträge in den Admin-Bereich.



4.2. Rollen und Berechtigungen

Drei Rollen steuern den Zugriff. Gäste sind nicht authentifiziert. Benutzer (`is_admin = 0`) dürfen eigene Kommentare erstellen, bearbeiten und löschen. Administratoren (`is_admin = 1`) verwalten zusätzlich Benutzer und Zitate und dürfen fremde Kommentare löschen, aber nicht bearbeiten.

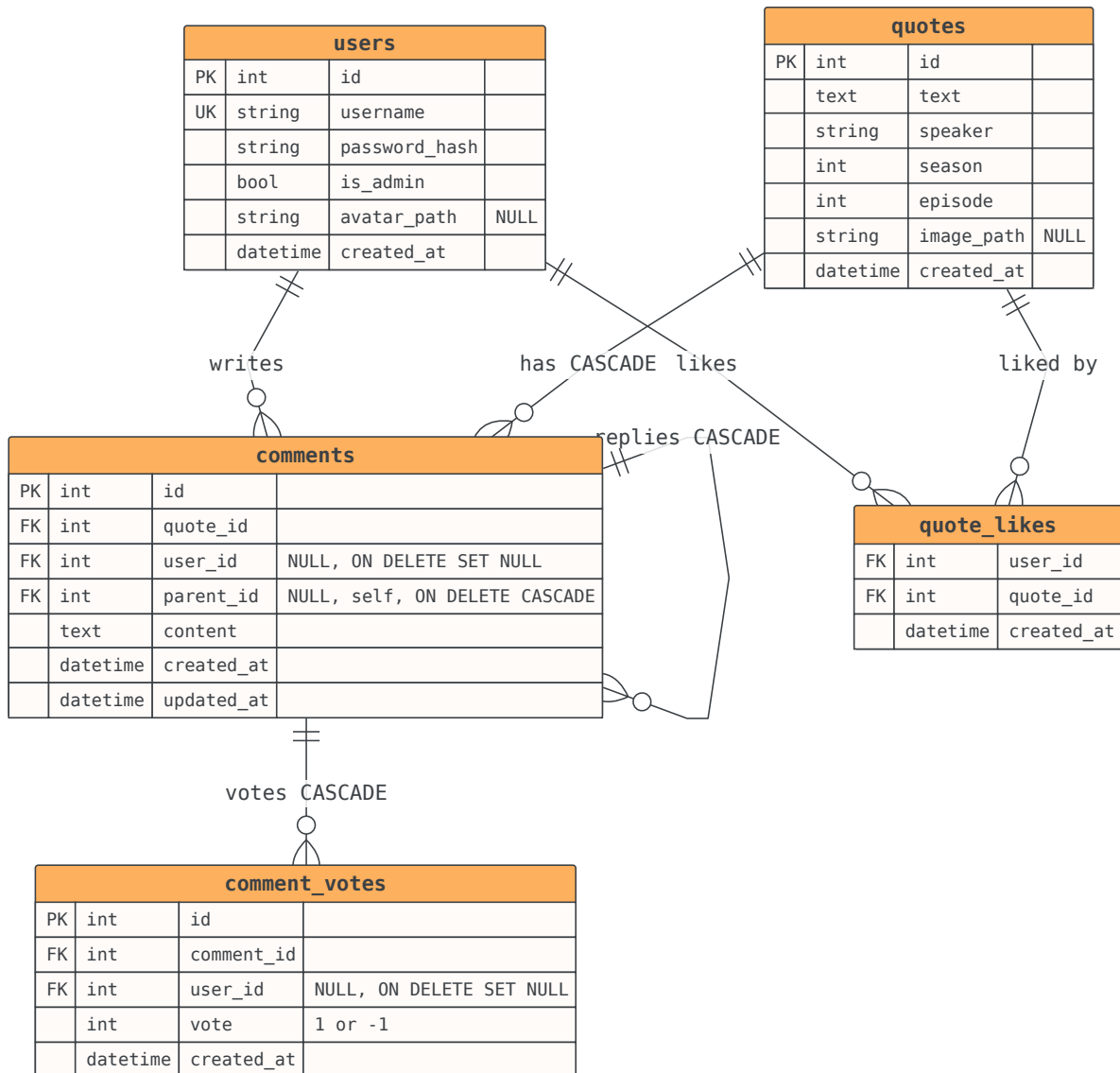
Aktion	Gast	User	Admin
Zitate und Kommentare lesen	✓	✓	✓
Registrierung und Login	✓	✓	✓
Kommentar schreiben, eigenes bearbeiten und löschen	-	✓	✓
Fremden Kommentar löschen	-	-	✓
Fremden Kommentar bearbeiten	-	-	X
Zitate verwalten (CRUD)	-	-	✓
Benutzerverwaltung	-	-	✓

Wird ein Benutzer gelöscht, bleiben seine Kommentare in der Datenbank erhalten. Der Fremdschlüssel `user_id` wird per `ON DELETE SET NULL` auf `NULL` gesetzt und in der Benutzeroberfläche als graues `<deleted>` angezeigt. So bleibt der Diskussionsverlauf nachvollziehbar.

5. Datenmodell

5.1. ER-Diagramm

Das folgende Entity-Relationship-Diagramm wurde manuell modelliert und visualisiert die Entitäten sowie deren Beziehungen. Es entspricht dem SQL-Schema in `WEB4_PHP_TEAM7.sql`.



5.2. Normalisierung und Constraints

Die Datenbank folgt den Grundregeln des relationalen Designs. Jede Entität besitzt einen numerischen Primärschlüssel. Benutzernamen sind eindeutig. Fremdschlüssel sichern referenzielle Integrität: Kommentare und Likes werden mit dem zugehörigen Zitat gelöscht (`CASCADE`), Nutzerreferenzen in Kommentaren und Votes werden bei Nutzerlöschung auf `NULL` gesetzt (`SET NULL`). Antworten referenzieren den Elternkommentar über `parent_id`; beim Löschen eines Kommentars mit Kindern greift `ON DELETE CASCADE` auf die gesamte Unterdiskussion.

5.2.1. users

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
username	VARCHAR(50)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL
is_admin	TINYINT(1)	NOT NULL, DEFAULT 0
avatar_path	VARCHAR(255)	NULL, Profilbild unter /uploads/avatars/
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

Passwörter werden ausschließlich als Hash gespeichert (`password_hash` via `password_hash()`). Klartextpasswörter existieren nicht in der Datenbank.

5.2.2. quotes

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
text	TEXT	NOT NULL
speaker	VARCHAR(100)	NOT NULL
season / episode	TINYINT UNSIGNED	NULL, optional
image_path	VARCHAR(255)	NULL, Beitragsbild unter /uploads/quotes/
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

5.2.3. comments

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
quote_id	INT UNSIGNED	FK → quotes.id, ON DELETE CASCADE
user_id	INT UNSIGNED	FK → users.id, NULL, ON DELETE SET NULL
parent_id	INT UNSIGNED	FK → comments.id, NULL, ON DELETE CASCADE
content	TEXT	NOT NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	NULL ON UPDATE

Kommentare sind die Haupt-CRUD-Ressource der Anwendung. `parent_id = NULL` kennzeichnet Top-Level-Kommentare, sonst handelt es sich um Thread-Antworten.

5.2.4. quote_likes

Spalte	Typ	Constraints
user_id	INT UNSIGNED	FK → users.id, ON DELETE CASCADE
quote_id	INT UNSIGNED	FK → quotes.id, ON DELETE CASCADE
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

Primärschlüssel ist das Paar `(user_id, quote_id)`. Pro Nutzer und Zitat ist höchstens ein Like möglich.

5.2.5. comment_votes

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
comment_id	INT UNSIGNED	FK → comments.id, ON DELETE CASCADE
user_id	INT UNSIGNED	FK → users.id, NULL, ON DELETE SET NULL
vote	TINYINT	NOT NULL, Werte 1 oder -1
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

5.3. Testdaten

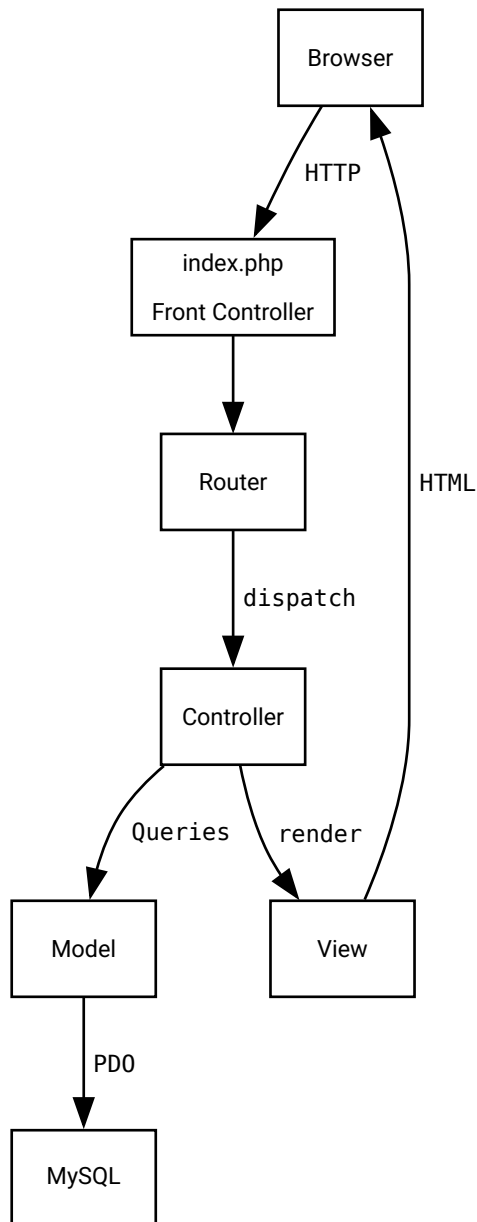
Der SQL-Dump `WEB4_PHP_TEAM7.sql` enthält das `CREATE DATABASE`-Statement, Tabellendefinitionen und repräsentative Testdaten. Nach dem Import stehen folgende Zugangsdaten zur Verfügung:

- Admin: `admin` / `admin`
- Testnutzer: `tyrion_fan`, `arya_fan` (Passwort jeweils `password123`)
- 12 Zitate, 17 Kommentare inklusive verschachtelter Antworten, Likes und Votes

6. Architektur

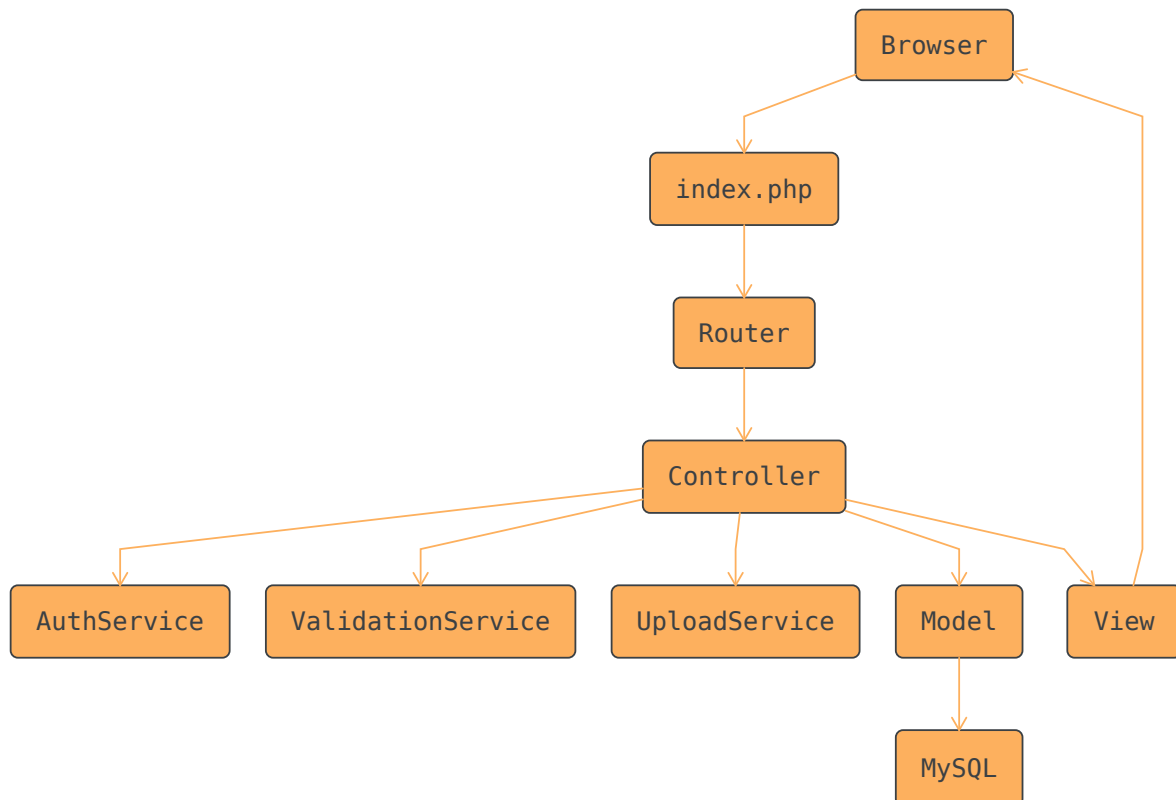
6.1. Schichtenmodell (MVC)

Die Projektangabe verlangt eine saubere Trennung der Anwendungsschichten. GoT Quotes implementiert klassisches MVC: Der Front Controller `index.php` leitet jede HTTP-Anfrage an den Router weiter. Controller orchestrieren Geschäftslogik und Berechtigungen, Models kapseln den PDO-Datenbankzugriff, Views rendern HTML mit escaped Ausgabe.



6.2. Komponentenübersicht

Innerhalb von `public/src/` sind die Verantwortlichkeiten klar getrennt. Services wie `AuthService`, `ValidationService` und `UploadService` enthalten querschnittliche Logik, die von mehreren Controllern genutzt wird.



6.3. Verzeichnisstruktur

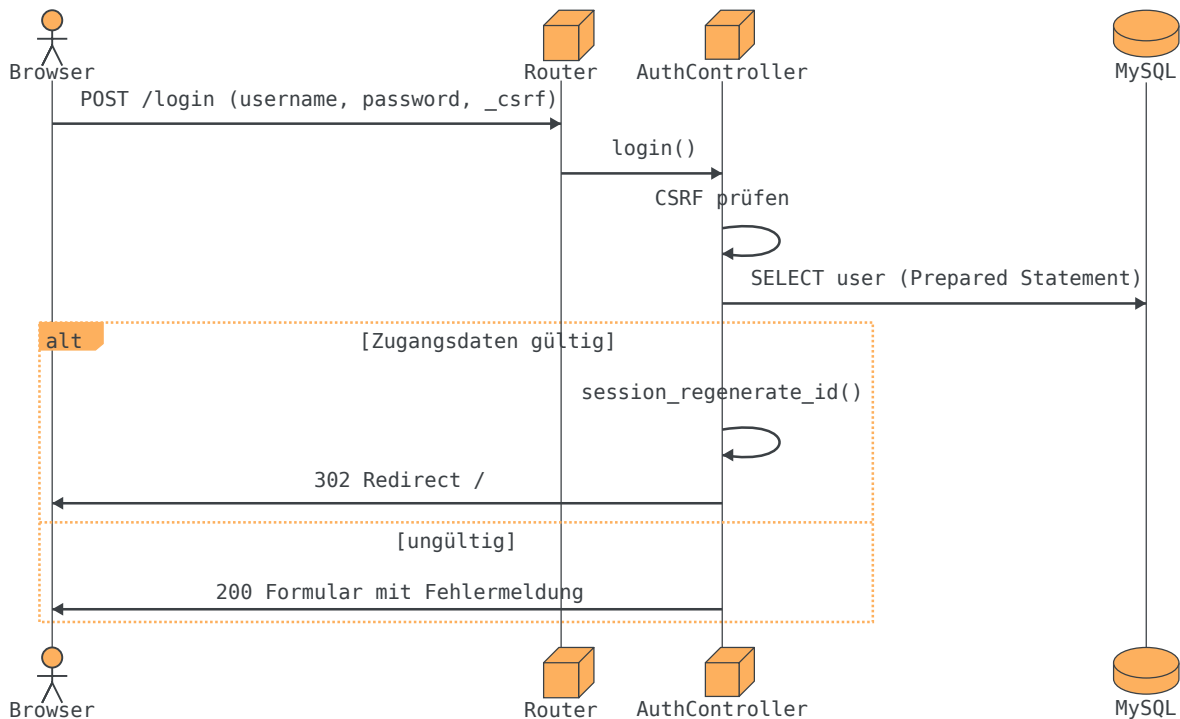
1	public/	
2	index.php	# Front Controller, REST-Routing, _method-Override
3	.htaccess	# URL Rewriting
4	assets/js/app.js	# Fetch für DELETE/PATCH, Thread-UI
5	uploads/	# Avatare und Zitatbilder (.htaccess ohne Script-Ausführung)
6	src/	
7	bootstrap.php	
8	config.php	
9	Router.php	# GET, POST, PUT, PATCH, DELETE
10	Controller/	# Auth, Profile, Quote, Comment, Admin/*
11	Model/	# User, Quote, Comment, QuoteLike, CommentVote
12	Service/	# AuthService, ValidationService, UploadService
13	views/	# PHP-Templates (Tailwind CDN)

Jede Schicht kennt nur die darunterliegende: Views enthalten kein SQL, Models enthalten keine HTTP-Logik. Der Router mappt Pfade und HTTP-Methoden auf Controller-Methoden und extrahiert Platzhalter wie `{id}`.

6.4. Ablauf: Login und Session

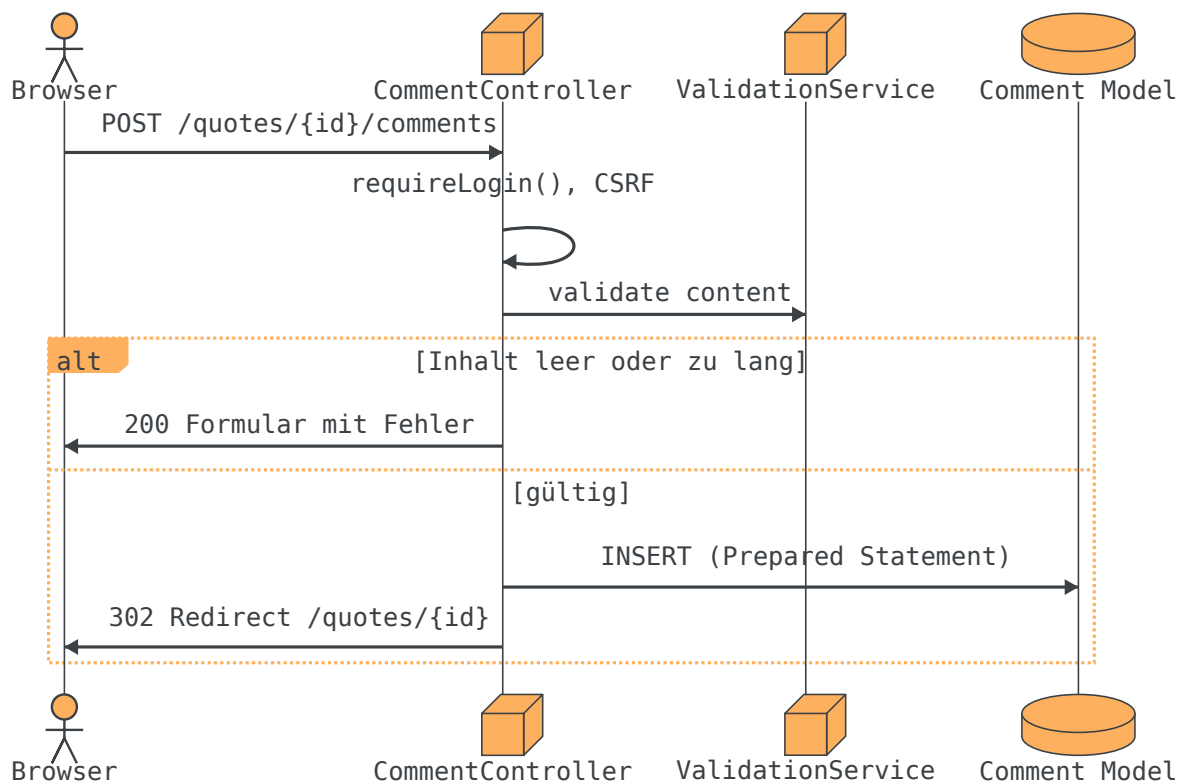
Authentifizierung erfolgt sessionbasiert. Nach erfolgreichem Login speichert PHP in `$_SESSION` die Felder `user_id`, `username`, `is_admin` und optional `avatar_path`.

`session_regenerate_id()` verhindert Session-Fixation. Flash-Messages informieren einmalig über Erfolg oder Fehler nach Redirects.



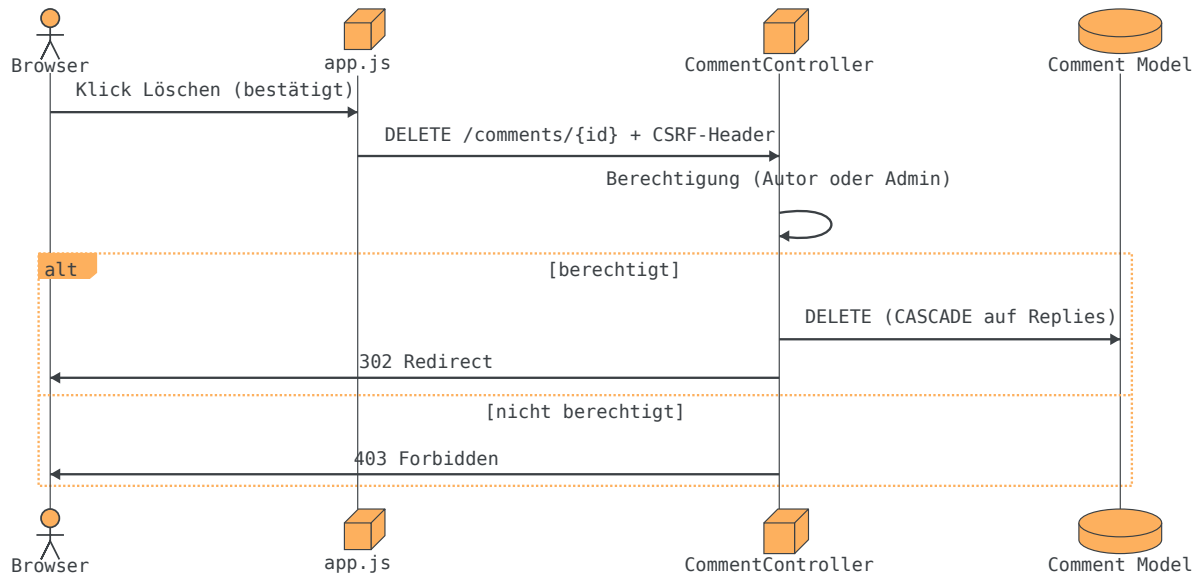
6.5. Ablauf: Kommentar anlegen (CRUD Create)

Das Erstellen eines Kommentars verdeutlicht die CRUD-Umsetzung für die Hauptressource. Der Controller prüft Login, CSRF und Validierung, bevor das Model den Datensatz einfügt.



6.6. Ablauf: Kommentar löschen per REST

Löschungen nutzen die HTTP-Methode DELETE und werden per JavaScript Fetch ausgelöst, da HTML-Formulare DELETE nicht nativ unterstützen. Der CSRF-Token wird als Header `X-CSRF-Token` mitgesendet.



6.7. REST-Routing

Die Anwendung verwendet ressourcenorientierte URLs und semantisch korrekte HTTP-Methoden. Es gibt keine Aktions-URLs wie `/comments/delete`. Browser-Formulare ohne Fetch nutzen bei PUT/PATCH/DELETE das versteckte Feld `_method`; `index.php` setzt daraus die effektive Request-Methode.

Methode	Pfad	Aktion
GET	/	Zitate-Feed mit <code>?sort=new top trending</code>
GET	/quotes/{id}	Zitat-Detail mit Kommentar-Thread
POST/DELETE	/quotes/{id}/likes	Zitat liken bzw. Like entfernen
POST/DELETE	/comments/{id}/votes	Kommentar bewerten
POST	/quotes/{id}/comments	Top-Level-Kommentar erstellen
POST	/comments/{id}/replies	Thread-Antwort erstellen
PUT	/comments/{id}	Eigenen Kommentar bearbeiten
DELETE	/comments/{id}	Kommentar löschen (Autor oder Admin)
GET	/register	Registrierungsformular
POST	/register	Neuen Benutzer anlegen
GET/POST	/login	Login-Formular bzw. Authentifizierung
POST	/logout	Session beenden
GET	/users/{id}	Öffentliches Profil

Methode	Pfad	Aktion
GET/PATCH/DELETE	/admin/users ...	Benutzerverwaltung
GET/POST/PUT/DELETE	/admin/quotes ...	Zitat-CRUD für Admins

7. Backend und Sicherheit

7.1. PDO und Prepared Statements

Sämtliche Datenbankzugriffe laufen über PDO mit Prepared Statements. User-Input wird nie per String-Konkatenation in SQL eingebettet. Die Model-Klassen kapseln alle Queries und geben gebundene Parameter an PDO weiter. Damit ist die Anwendung gegen SQL-Injection abgesichert.

7.2. Validierung

Jeder textuelle Input aus Formularen oder Uploads wird serverseitig geprüft. Der `ValidationService` setzt unter anderem folgende Regeln durch:

Feld	Regeln
username	3 bis 50 Zeichen, nur <code>[a-zA-Z0-9_]</code> , eindeutig
password	mindestens 8 Zeichen
comment.content	1 bis 1000 Zeichen, nicht leer nach <code>trim()</code>
quote.text	1 bis 2000 Zeichen
quote.speaker	1 bis 100 Zeichen
Bild-Uploads	optional, max. 2 MB, JPEG/PNG/WebP, MIME-Check via <code>finfo</code>

Fehlerhafte Eingaben führen nicht zu stillschweigendem Verwerfen: Das Formular wird erneut angezeigt, Fehlermeldungen erscheinen am betroffenen Feld oder als Flash-Message. Auch in den automatisierten Tests (`cmtEmptyRejected`, `authLoginFail`) werden fehlerhafte Eingaben explizit geprüft.

7.3. Schutzmaßnahmen im Überblick

Bedrohung	Maßnahme
SQL Injection	PDO Prepared Statements für alle Queries mit User-Input
XSS	<code>htmlspecialchars()</code> bei jeder Ausgabe usergenerierter Inhalte
Session Fixation	<code>session_regenerate_id()</code> nach Login
CSRF	Token in POST-Formularen und Fetch-Header <code>X-CSRF-Token</code>
IDOR	Bearbeitung nur für Kommentar-Autor; Admin darf fremde Kommentare nur löschen

Bedrohung	Maßnahme
Passwörter	<code>password_hash()</code> / <code>password_verify()</code> , <code>PASSWORD_DEFAULT</code>
Uploads	MIME-Check, Größenlimit, zufällige Dateinamen, <code>.htaccess</code> ohne Script-Ausführung

8. Frontend und Benutzeroberfläche

8.1. HTML, CSS und JavaScript

Die Oberfläche besteht aus validem HTML5. Formatierungen erfolgen ausschließlich über CSS, konkret Tailwind CSS 4 per CDN im Layout-Template. Die Navigation führt Gäste zu Feed, Login und Registrierung, eingeloggte Nutzer zusätzlich zu Profil und Logout, Administratoren zu den Admin-Bereichen Benutzer und Zitate.

JavaScript ist bewusst auf Transport- und Komfortfunktionen beschränkt. Die Datei `public/assets/js/app.js` sendet per Fetch API DELETE- und PATCH-Anfragen, übergibt CSRF-Token aus dem Meta-Tag und blendet Antwort-Formulare im Thread ein. Validierung, Berechtigungsprüfung und Escaping bleiben vollständig serverseitig, wie von der Projektangabe gefordert.

8.2. Foren- und Thread-Darstellung

Zitate erscheinen im Feed als Karten mit Ausschnitt, Sprecher, optional Thumbnail, Kommentar- und Like-Zähler. Auf der Detailseite zeigt ein Hero-Bild (falls vorhanden) das vollständige Zitat. Kommentare werden rekursiv in `comment-tree.php` gerendert: Avatare links, Einrückung und Verbindungslinien visualisieren die Thread-Struktur. Sortierung ist sowohl im Feed (`?sort=`) als auch bei Kommentaren (`?csort=`) wählbar.

9. Testfälle

Die Projektangabe verlangt ausführliche Tests der Anwendung, einschließlich fehlerhafter Eingaben. Wir unterscheiden automatisierte REST-Tests (httpYac) und manuelle UI-Tests in Google Chrome unter XAMPP.

9.1. Automatisierter REST-Testlauf (httpYac)

Die REST-Schnittstelle wird mit httpYac getestet. Jede `.rest`-Datei beschreibt HTTP-Anfragen mit Assertions (`?? status = 200`, `?? body includes ...`). Der Lauf erzeugt einen JUnit-Report, den diese Dokumentation einliest.

Schritt	Befehl bzw. Pfad
Tests ausführen	<code>mise run test:rest</code>
JUnit-Report	<code>doc/test-results/httpyac-junit.xml</code>
Testdateien	<code>tests/rest/01-public.rest</code> bis <code>05-forum.rest</code>
Voraussetzung	App unter <code>http://127.0.0.1:80</code> , SQL-Dump importiert

Die Dateien folgen einem einheitlichen Namensschema (`pub*`, `auth*`, `cmt*`, `adm*`, `forum*`). Fehlerfälle sind darin enthalten, etwa Login mit falschem Passwort, leerer Kommentar, Zugriff auf Admin ohne Rolle und DELETE ohne CSRF.

Die folgenden Tabellen wurden aus dem JUnit-Report generiert. Spalte **Erwartet** stammt aus der Assertion in der `.rest`-Datei, **Beobachtet** ist der tatsächliche Wert.

9.1.1. Übersicht REST-Tests

Testsuite	Tests	Fehler
Total	47	0
\tests\rest\01-public.rest	11	0
\tests\rest\02-auth.rest	11	0
\tests\rest\03-comments.rest	11	0
\tests\rest\04-admin.rest	7	0
\tests\rest\05-forum.rest	7	0

9.1.2. Erwartete und beobachtete Werte

Request	Prüfung	Beobachtet	Status
\tests\rest\01-public.rest			
pubFeed	status == 200	200	OK
pubFeed	body contains Zitate		OK
pubQuoteDetail	status == 200	200	OK
pubQuoteDetail	body contains Cersei Lannister		OK
pubQuote404	status == 404	404	OK
pubProfile	status == 200	200	OK
pubProfile	body contains tyrior_fan		OK
pubSortTop	status == 200	200	OK
pubSortTop	body contains sort=top		OK
pubCommentSort	status == 200	200	OK
pubAdminForbidden	status == 403	403	OK
\tests\rest\02-auth.rest			
authLoginForm	status == 200	200	OK
authLoginForm	body contains Einloggen		OK
authLoginOk	status == 302	302	OK
authLoginOk	location == /	/	OK
authSessionCheck	status == 200	200	OK
authSessionCheck	body contains tyrior_fan		OK
authLogout	status == 302	302	OK
authProfileRedirect	status == 302	302	OK
authProfileRedirect	location == /login	/login	OK

Request	Prüfung	Beobachtet	Status
\tests\rest\02-auth.rest			
authLoginFail	status == 200	200	OK
authLoginFail	body contains ungültig		OK
\tests\rest\03-comments.rest			
cmtCreate	status == 302	302	OK
cmtShow	status == 200	200	OK
cmtShow	body contains Automatisierter REST-Test		OK
cmtForbiddenEdit	status == 403	403	OK
cmtEditForm	status == 200	200	OK
cmtEditForm	body contains Kommentar bearbeiten		OK
cmtUpdate	status == 302	302	OK
cmtUpdated	status == 200	200	OK
cmtUpdated	body contains REST updated		OK
cmtEmptyRejected	status == 200	200	OK
cmtEmptyRejected	body contains darf nicht leer		OK
\tests\rest\04-admin.rest			
admUserList	status == 200	200	OK
admUserList	body contains Benutzer		OK
admQuoteList	status == 200	200	OK
admQuoteList	body contains Zitate		OK
admQuoteNewForm	status == 200	200	OK
admQuoteNewForm	body contains Anlegen		OK
admUserForbidden	status == 403	403	OK
\tests\rest\05-forum.rest			
forumLike	status == 302	302	OK
forumUnlike	status == 302	302	OK
forumUpvote	status == 302	302	OK
forumRemoveVote	status == 302	302	OK
forumDeleteNoCsrf	status == 302	302	OK
forumDeleteNoCsrf	location == /	/	OK
forumBadLike	status == 404	404	OK

9.2. Manuelle UI-Tests

Zusätzlich zu den automatisierten Tests wurden Szenarien manuell geprüft. Dazu gehören Registrierung und Validierung, CRUD an Kommentaren, Admin-Funktionen, Foren-Interaktionen (Likes, Votes, Sortierung), Bild-Uploads, responsives Layout und Sicherheitsfälle wie XSS-Escaping und IDOR-Schutz.

9.2.1. Übersicht manuelle Tests

Testsuite	Tests	Fehler
Total	32	0
Authentifizierung	6	0
Zitate & Kommentare	9	0
Administration	8	0
Forum & Interaktion	6	0
Sicherheit	3	0

9.2.2. Detailergebnisse manuelle Tests

Testname	Status	Zeit
Authentifizierung		
T-AUTH-01: Registrierung gültig → Erfolg, Redirect Login	✓ Erfolgreich	manuell
T-AUTH-02: Duplicate Username → Fehler	✓ Erfolgreich	manuell
T-AUTH-03: Passwort < 8 Zeichen → Fehler	✓ Erfolgreich	manuell
T-AUTH-04: Login korrekt → Session aktiv	✓ Erfolgreich	manuell
T-AUTH-05: Login falsch → generische Fehlermeldung	✓ Erfolgreich	manuell
T-AUTH-06: Logout → Session beendet	✓ Erfolgreich	manuell
Zitate & Kommentare		
T-QUOTE-01: Gast sieht Liste und Detail	✓ Erfolgreich	manuell
T-CMT-01: Gast sieht Kommentare, kein Formular	✓ Erfolgreich	manuell
T-CMT-02: User erstellt Kommentar mit Username	✓ Erfolgreich	manuell
T-CMT-03: User bearbeitet eigenen Kommentar	✓ Erfolgreich	manuell
T-CMT-04: User löscht eigenen Kommentar	✓ Erfolgreich	manuell
T-CMT-05: Fremdes Edit → 403	✓ Erfolgreich	manuell
T-CMT-06: Fremdes Delete → 403	✓ Erfolgreich	manuell
T-CMT-07: Leerer Kommentar → Validierungsfehler	✓ Erfolgreich	manuell
T-CMT-08: XSS-Payload → escaped in Ausgabe	✓ Erfolgreich	manuell
Administration		
T-ADM-01: Admin löscht fremden Kommentar	✓ Erfolgreich	manuell
T-ADM-02: Admin kann fremden Kommentar nicht bearbeiten	✓ Erfolgreich	manuell
T-ADM-03: Admin togglet User-Rolle	✓ Erfolgreich	manuell
T-ADM-04: Admin löscht User → Kommentare zeigen <deleted>	✓ Erfolgreich	manuell
T-ADM-05: Nicht-Admin auf /admin → 403	✓ Erfolgreich	manuell
T-ADM-06: Letzter Admin nicht degradierbar	✓ Erfolgreich	manuell
T-ADM-07: Admin CRUD Zitate	✓ Erfolgreich	manuell
T-ADM-08: Admin kann eigene Rolle nicht entziehen	✓ Erfolgreich	manuell

Testname	Status	Zeit
Forum & Interaktion		
T-FORUM-01: Feed-Sortierung top/trending	✓ Erfolgreich	manuell
T-FORUM-02: Zitat liken und entliken	✓ Erfolgreich	manuell
T-FORUM-03: Kommentar up-/downvoten	✓ Erfolgreich	manuell
T-FORUM-04: Kommentar-Sortierung auf Detailseite	✓ Erfolgreich	manuell
T-FORUM-05: Öffentliches Profil mit Kommentaren und Likes	✓ Erfolgreich	manuell
T-FORUM-06: Thread-Antworten und CASCADE-Löschung	✓ Erfolgreich	manuell
Sicherheit		
T-SEC-01: SQL-Injection Login → kein Bypass	✓ Erfolgreich	manuell
T-SEC-02: Direktaufruf fremdes Edit → 403	✓ Erfolgreich	manuell
T-SEC-03: Ungültige Quote-ID → 404	✓ Erfolgreich	manuell

10. Installation und Abgabe

10.1. Voraussetzungen

Die Anwendung ist für eine Standard-XAMPP-Installation unter Windows vorgesehen (Apache, PHP 8.x, MySQL/MariaDB, Google Chrome als Testbrowser). Es ist kein Composer, npm oder Build-Schritt nötig: Das entpackte ZIP genügt zum Betrieb.

10.2. Installationsschritte

1. ZIP-Archiv `WEB4_PHP_TEAM7.zip` entpacken
2. In phpMyAdmin den SQL-Dump `WEB4_PHP_TEAM7.sql` importieren (enthält `CREATE DATABASE`)
3. Apache DocumentRoot auf `public/` setzen
4. Anwendung unter `http://localhost/` aufrufen
5. Optional mit `admin` / `admin` einloggen und Admin-Bereich prüfen

10.3. Datenbankzugang

Parameter	Wert
Datenbankname	<code>team_7</code>
Benutzername	<code>fh_webphp</code>
Passwort	<code>fh_webphp</code>
SQL-Dump	<code>WEB4_PHP_TEAM7.sql</code> (Root des Repositories)

Die Konfiguration liegt in `public/src/config.php` bzw. optional `config.local.php`. Vor der Abgabe wurde geprüft, dass der Dump ohne Fehler importiert werden kann und die Anwendung danach unter der dokumentierten Start-URL erreichbar ist.

10.4. Abgabeformat

Gemäß Projektangabe wird ein ZIP-Archiv im Schema `WEB4_PHP_TEAM7.zip` abgegeben. Enthalten sind der Anwendungscode unter `public/`, die Dokumentation als PDF unter `doc/`,

der SQL-Dump und die REST-Testdateien. Das PDF entsteht aus dieser Typst-Quelle mit `mise`
`run build:documentation`.